

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT on

Artificial Intelligence (23CS5PCAIN)

Submitted by

Sameksha (1BM23CS292)

in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Aug 2025 to Dec 2025

B.M.S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the Lab work entitled “Artificial Intelligence (23CS5PCAIN)” carried out by **Sameksha (1BM23CS292)**, who is bonafide student of **B.M.S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum. The Lab report has been approved as it satisfies the academic requirements in respect of an Artificial Intelligence (23CS5PCAIN) work prescribed for the said degree.

Mrs.Seema Patil Assistant Professor Department of CSE, BMSCE	Dr. Kavitha Sooda Professor & HOD Department of CSE, BMSCE
--	--

Index

Sl. No.	Date	Experiment Title	Page No.
1	30-9-2024	Implement Tic –Tac –Toe Game Implement vacuum cleaner agent	5
2	7-10-2024	Implement 8 puzzle problems using Depth First Search (DFS) Implement Iterative deepening search algorithm	15
3	14-10-2024	Implement A* search algorithm	26
4	21-10-2024	Implement Hill Climbing search algorithm to solve N-Queens problem	32
5	28-10-2024	Simulated Annealing to Solve 8-Queens problem	38
6	11-11-2024	Create a knowledge base using propositional logic and show that the given query entails the knowledge base or not.	40
7	2-12-2024	Implement unification in first order logic	44
8	2-12-2024	Create a knowledge base consisting of first order logic statements and prove the given query using forward reasoning.	48
9	16-12-2024	Create a knowledge base consisting of first order logic statements and prove the given query using Resolution	55
10	16-12-2024	Implement Alpha-Beta Pruning.	61

https://github.com/sameksha-1bm23cs292/artificial_intelligence

4

Program 1

Implement Tic - Tac - Toe Game

Algorithm:

Week 1 18/08/25

Lab 1

Implement: Tic - Tac - Toe game

Initial state:

X	0	X
	0	
0		X

Game tree branches:

1. X wins

X	0	X
	0	X
X		X

2. 0 wins

X	0	X
X	0	
0		X

3. draw

X	0	X
	0	
0	X	X

4. draw

X	0	X
X	0	
0	X	

5. draw

X	0	X
	0	
0	X	

6. draw

X	0	X
X	0	
0	X	

7. draw

X	0	X
	0	
0	X	

Algorithm

Step 1: Start the game.

Step 2: ask for letter X and O from players.

Step 3: decide player 1 or player 2 to go first.

Step 4: ask player 1 to place his letter in the check box.

- step 5: check if that wins.
- step 6: check for draw.
- step 7: ask for player 2 to place his letter in the check box.
- step 8: check if that wins.
- step 9: check for draw.
- step 10: ask for player to play again if there is any chance to win.
- step 11: end

Output :

Player x's turn

Enter row (1, 2, or 3) : 1

Enter column (1, 2, or 3) : 1

X		

Player O's turn

Enter row (1, 2, or 3) : 1

column (1, 2, or 3) : 2

X	O	

Player X's turn

Enter row (1, 2, or 3): 1

Enter column (1, 2, or 3): 3

X	O	X

Player O's turn

Enter row (1, 2, or 3): 2

Enter column (1, 2, or 3): 2

X	O	X
	O	

Player X's turn

Enter row (1, 2, or 3): 3

Enter column (1, 2, or 3): 3

X	O	X
	O	
		X

Player O's turn

row: 3

column: 1

X	O	X
	O	
O		X

~~Player X's turn~~

X	O	X
	O	X
O		X

Player X wins!

Code:

```
#week 1 TIC TAC TOE
board = [ ["-", "-", "-"], ["-", "-", "-"], ["-", "-", "-"] ]
Xrow = {"0":0, "1":0, "2":0}
Orow = {"0":0, "1":0, "2":0}
Xcol = {"0":0, "1":0, "2":0}
Ocol = {"0":0, "1":0, "2":0}
win = False

def checkWin():
    global win
    for key in Xrow:
        if Xrow[key] == 3:
            win = True
    for key in Xcol:
        if Xcol[key] == 3:
            win = True
    for key in Orow:
        if Orow[key] == 3:
            win = True
    for key in Ocol:
        if Ocol[key] == 3:
            win = True
    if board[0][0] == board[1][1] == board[2][2] != "-":
        win = True
    if board[0][2] == board[1][1] == board[2][0] != "-":
        win = True

def place(symbol, row, col):
    if board[row][col] != "-":
        print("Can't place at this spot, try again")
        return False

    if symbol == "1":
        board[row][col] = "X"
        Xrow[str(row)] += 1
        Xcol[str(col)] += 1
    elif symbol == "2":
        board[row][col] = "O"
        Orow[str(row)] += 1
        Ocol[str(col)] += 1

    return True

def displayBoard():
    for row in board:
```

```

        print(row)
    print("\n")

# Game loop
displayBoard()
switch = input("Enter 1 for X, 2 for O to start: ")

while any("-" in row for row in board) and not win:
    try:
        row = int(input("Enter Row (1-3): "))
        col = int(input("Enter Column (1-3): "))
    except ValueError:
        print("Invalid input, enter numbers only.")
        continue

    if row > 3 or col > 3 or row < 1 or col < 1:
        print("Invalid input, please try again.")
        continue

    decision = place(switch, row - 1, col - 1)
    if decision:
        displayBoard()
        checkWin()
        if not win:
            switch = "2" if switch == "1" else "1"

if win:
    winner = "X" if switch == "1" else "O"
    print(f"{winner} wins!")
else:
    print("It's a draw!")

print("Sameksha 1BM23CS292")

```

Output:

```

['-', '-', '-']
['-', '-', '-']
['-', '-', '-']

```

```

Enter 1 for X, 2 for O to start: 1
Enter Row (1-3): 1
Enter Column (1-3): 1
['X', '-', '-']
['-', '-', '-']
['-', '-', '-']

```

```
Enter Row (1-3): 2
Enter Column (1-3): 3
['X', '-', '-']
['-', '-', 'O']
['-', '-', '-']
```

```
Enter Row (1-3): 1
Enter Column (1-3): 2
['X', 'X', '-']
['-', '-', 'O']
['-', '-', '-']
```

```
Enter Row (1-3): 3
Enter Column (1-3): 3
['X', 'X', '-']
['-', '-', 'O']
['-', '-', 'O']
```

```
Enter Row (1-3): 2
Enter Column (1-3): 1
['X', 'X', '-']
['X', '-', 'O']
['-', '-', 'O']
```

```
Enter Row (1-3): 3
Enter Column (1-3): 1
['X', 'X', '-']
['X', '-', 'O']
['O', '-', 'O']
```

```
Enter Row (1-3): 1
Enter Column (1-3): 3
['X', 'X', 'X']
['X', '-', 'O']
['O', '-', 'O']
```

```
X wins!
Sameksha 1BM23CS292
```

Implement vacuum cleaner agent

25/08/25

Date ____
Page ____

Lab 2

Implement Vacuum cleaner.

Algorithm:

Step 1: Start

Step 2: Take the phones from user either Right, Left, Sweep (suck)

Step 3: make the move. If dust is present when sweep is called. do the action

Step 1: Start

Step 2: vacuum cleaner is present in Room 1. Enter the action Right, Left or Sweep.

Right → vacuum moves to right side

Left → Vacuum moves to left side

Sweep → the vacuum sucks the dust

Step 3: when vacuum enters room 2

Ask again for the actions and do the respective rooms.

Step 4: To exit the program enter END

Output :

Enter initial state for room A (1 for dirty, 0 for clean) : 1

Enter initial state for room B (1 for dirty, 0 for clean) : 0

Starting in room : Room X

Initial room status : { 'A' : 'dirty', 'B' : 'clean' }

Available actions: sweep, right, left
 Enter action: right
 Cannot move right from room B. Staying in the same room.
 Available actions: sweep, right, left
 Enter action: left
 Moved left to room: A
 Available actions: sweep, right, left
 Enter action: sweep
 Swept room A. It is clean now.

All rooms are clean.
 Task complete.
 Total cost: 1

Handwritten signature and date: 25/12/20

#week 1 VACCUUM CLEANER

import random

class VacuumCleanerAgent:

def __init__(self, rooms):

self.rooms = rooms

self.current_room = random.choice(list(rooms.keys()))

self.cost = 0

print(f"Starting in room: {self.current_room}")

print(f"Initial room states: {self.rooms}")

def sweep(self):

if self.rooms[self.current_room] == 'dirty':

self.rooms[self.current_room] = 'clean'

self.cost += 1 # Increment cost for sweeping

print(f"Swept room {self.current_room}. It is now clean.")

return True

else:

print(f"Room {self.current_room} is already clean.")


```

        return False

    def right(self):
        room_names = list(self.rooms.keys())
        current_index = room_names.index(self.current_room)
        if current_index < len(room_names) - 1:
            self.current_room = room_names[current_index + 1]
            print(f"Moved right to room: {self.current_room}")
        else:
            print(f"Cannot move right from room {self.current_room}. Staying
in the same room.")

    def left(self):
        room_names = list(self.rooms.keys())
        current_index = room_names.index(self.current_room)
        if current_index > 0:
            self.current_room = room_names[current_index - 1]
            print(f"Moved left to room: {self.current_room}")
        else:
            print(f"Cannot move left from room {self.current_room}. Staying
in the same room.")

    def perceive(self):
        return self.rooms[self.current_room]

initial_states = {}
for room in rooms.keys():
    while True:
        state = input(f"Enter initial state for room {room} (1 for dirty, 0
for clean): ")
        if state == '1':
            initial_states[room] = 'dirty'
            break
        elif state == '0':
            initial_states[room] = 'clean'
            break
        else:
            print("Invalid input. Please enter 1 or 0.")

agent = VacuumCleanerAgent(initial_states)
while any(status == 'dirty' for status in agent.rooms.values()):
    print("\nAvailable actions: sweep, right, left")
    action = input("Enter action: ").lower()

```

```

    if action == 'sweep':
        agent.sweep()
    elif action == 'right':
        agent.right()
    elif action == 'left':
        agent.left()
    else:
        print("Invalid action. Please enter sweep, right, or left.")

print("\nAll rooms are clean. Task complete.")
print(f"Total cost: {agent.cost}")
print("Sameksha 1BM23CS292")

```

output:

```

Enter initial state for room A (1 for dirty, 0 for clean): 1
Enter initial state for room B (1 for dirty, 0 for clean): 0
Starting in room: B
Initial room states: {'A': 'dirty', 'B': 'clean'}

Available actions: sweep, right, left
Enter action: right
Cannot move right from room B. Staying in the same room.

Available actions: sweep, right, left
Enter action: left
Moved left to room: A

Available actions: sweep, right, left
Enter action: left
Cannot move left from room A. Staying in the same room.

Available actions: sweep, right, left
Enter action: sweep
Swept room A. It is now clean.

All rooms are clean. Task complete.
Total cost: 1
Sameksha 1BM23CS292

```

PROGRAM 2

Implement 8 puzzle problems using Depth First Search (DFS)

mlk 2 01/09/25

Implement using BFS solve 8 puzzle
'without' heuristic.

2	8	3
1	6	4
7		5

initial

1	2	3
8		4
7	5	6

goal.

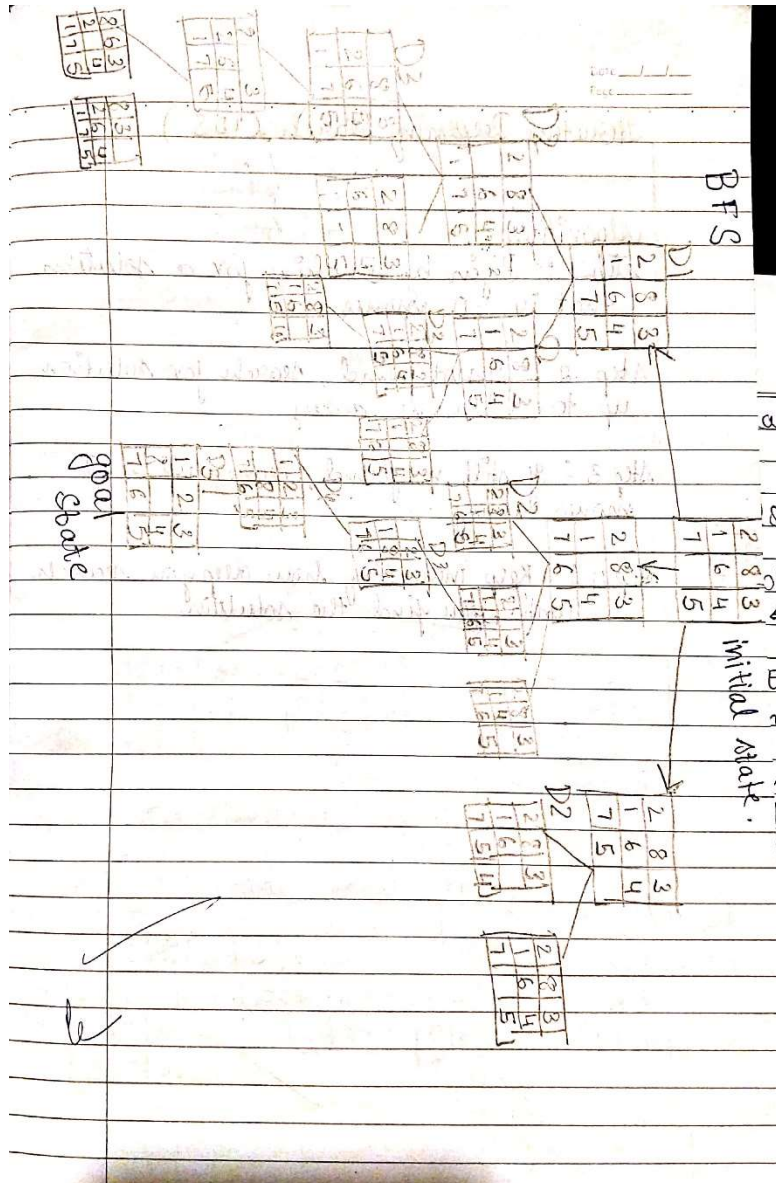
Algorithm :

- Step 1: Start
- Step 2: Enter initial state followed by goal state.
- Step 3: move the empty box in all possible ways (either up, down, left or right) and display it.
- Step 4: repeat the same step for each displayed unit and check with the goal state.
- Step 5: If it matches the goal state, ~~it~~ unit and display it.
else continue displaying possible moves.
- Step 6: Exit.

OUTPUT :

Solution found !

Moves : 5



```
#WEEK2 8PUZZLE PPROBLEM in BFS
from collections import deque

goal_state = '123804765'
# Using '0' as the empty space

# Moves: U, D, L, R (Up, Down, Left, Right)
moves = {
    'U': -3,
    'D': 3,
    'L': -1,
```

```

        'R': 1
    }

# Valid moves for each index to prevent wrapping around rows/columns
invalid_moves = {
    0: ['U', 'L'], 1: ['U'], 2: ['U', 'R'],
    3: ['L'],      5: ['R'],
    6: ['D', 'L'], 7: ['D'], 8: ['D', 'R']
}

# Function to generate new puzzle state after moving the blank
def move_tile(state, direction):
    index = state.index('0')
    if direction in invalid_moves.get(index, []):
        return None

    new_index = index + moves[direction]
    if new_index < 0 or new_index >= 9:
        return None

    state_list = list(state)
    # Swap 0 with the target tile
    state_list[index], state_list[new_index] = state_list[new_index],
state_list[index]
    return ''.join(state_list)

# Helper function to print the state in 3x3 format
def print_state(state):
    for i in range(0, 9, 3):
        # Replace '0' with space for readability
        print(' '.join(state[i:i+3]).replace('0', ' '))
    print() # Blank line for spacing

# BFS Algorithm with printing all visited states
def bfs(start_state):
    visited = set()
    queue = deque([(start_state, [])]) # Each element is (state, path)

    while queue:
        current_state, path = queue.popleft()

        if current_state in visited:
            continue

        # Print each visited state
        print("Visited state:")

```

```

    print_state(current_state)

    if current_state == goal_state:
        return path

    visited.add(current_state)

    for direction in moves:
        new_state = move_tile(current_state, direction)
        if new_state and new_state not in visited:
            queue.append((new_state, path + [direction]))

    return None # No solution found

# Input initial state as a string (e.g., '123456780' where 0 is the blank)
start = input("Enter the INITIAL state (give '0' for empty space): ")

if len(start) == 9 and set(start) == set('012345678'):
    print("INITIAL state:")
    print_state(start)

    result = bfs(start)

    if result is not None:
        print("Solution found!")
        print("Moves:", ' '.join(result))
        print("Number of moves:", len(result))
        current_state = start
        for i, move in enumerate(result, 1):
            current_state = move_tile(current_state, move)
            print(f"Move {i}: {move}")
            print_state(current_state)
    else:
        print("No solution exists for the given start state.")
else:
    print("Invalid input! Please enter a 9-digit string using digits 0-8 without repetition.")
print("1BM23CS292 Sameksha\n")

```

OUTPUT:

```

Enter the INITIAL state (give '0' for empty space): 283164705
INITIAL state:
2 8 3
1 6 4
7   5

Visited state:
2 8 3

```


1 6 4
7 5

Visited state:
2 8 3
1 4
7 6 5

Visited state:
2 8 3
1 6 4
7 5

Visited state:
2 8 3
1 6 4
7 5

Visited state:
2 3
1 8 4
7 6 5

Visited state:
2 8 3
1 4
7 6 5

Visited state:
2 8 3
1 4
7 6 5

Visited state:
2 8 3
6 4
1 7 5

Visited state:
2 8 3
1 6
7 5 4

Visited state:
2 3
1 8 4
7 6 5

Visited state:
2 3
1 8 4
7 6 5

Visited state:
8 3
2 1 4
7 6 5

Visited state:
2 8 3
7 1 4
6 5

Visited state:
2 8
1 4 3
7 6 5

Visited state:
2 8 3
1 4 5
7 6

Visited state:
8 3
2 6 4
1 7 5

Visited state:
2 8 3
6 4
1 7 5

Visited state:
2 8
1 6 3
7 5 4

Visited state:
2 8 3
1 6
7 5 4

Visited state:
1 2 3
8 4
7 6 5

Visited state:
2 3 4
1 8
7 6 5

Visited state:
8 3
2 1 4
7 6 5

Visited state:
2 8 3
7 1 4
6 5

Visited state:
2 8
1 4 3

7 6 5

Visited state:

2 8 3

1 4 5

7 6

Visited state:

8 3

2 6 4

1 7 5

Visited state:

2 3

6 8 4

1 7 5

Visited state:

2 8 3

6 7 4

1 5

Visited state:

2 8 3

6 4

1 7 5

Visited state:

2 8

1 6 3

7 5 4

Visited state:

2 3

1 8 6

7 5 4

Visited state:

2 8 3

1 5 6

7 4

Visited state:

2 8 3

1 6

7 5 4

Visited state:

1 2 3

7 8 4

6 5

Visited state:

1 2 3

8 4

7 6 5

Solution found!

Moves: U U L D R
Number of moves: 5
Move 1: U
2 8 3
1 4
7 6 5

Move 2: U
2 3
1 8 4
7 6 5

Move 3: L
2 3
1 8 4
7 6 5

Move 4: D
1 2 3
8 4
7 6 5

Move 5: R
1 2 3
8 4
7 6 5

1BM23CS292 Sameksha

Implement Iterative deepening search algorithm

Date / /
Page

Iterative Deepening Search (IDS)

Algorithm:

Step 1: Begin by searching for a solution that is 0 moves

Step 2: If not found, search for solution up to 1 move away

Step 3: If still not found, search up to 2 moves away.

Step 4: Keep increasing how deep you search until you find the solution

1	2	3
4	0	5
6	7	8

1	2	3
4	5	0
6	7	8

Initial Goal

deep limit = 1

1	2	3
0	4	5
6	7	8

right

1	2	3
4	5	0
6	7	8

up

1	0	3
4	2	5
6	7	8

down

1	2	3
4	7	5
6	0	8

#WEEK2 8PUZZLE PPROBLEM in DFS

goal_state = '123804765'

moves = {

```

        'U': -3,
        'D': 3,
        'L': -1,
        'R': 1
    }

    invalid_moves = {
        0: ['U', 'L'], 1: ['U'], 2: ['U', 'R'],
        3: ['L'],       5: ['R'],
        6: ['D', 'L'], 7: ['D'], 8: ['D', 'R']
    }

def move_tile(state, direction):
    index = state.index('0')
    if direction in invalid_moves.get(index, []):
        return None

    new_index = index + moves[direction]
    if new_index < 0 or new_index >= 9:
        return None

    state_list = list(state)
    state_list[index], state_list[new_index] = state_list[new_index],
state_list[index]
    return ''.join(state_list)

def print_state(state):
    for i in range(0, 9, 3):
        print(' '.join(state[i:i+3]).replace('0', ' '))
    print()

def dfs(start_state, max_depth=50):
    visited = set()
    stack = [(start_state, [])] # Each element: (state, path)

    while stack:
        current_state, path = stack.pop()

        if current_state in visited:
            continue

        # Print every visited state
        print("Visited state:")
        print_state(current_state)

        if current_state == goal_state:

```



```

        return path

    visited.add(current_state)

    if len(path) >= max_depth:
        continue

    for direction in moves:
        new_state = move_tile(current_state, direction)
        if new_state and new_state not in visited:
            stack.append((new_state, path + [direction]))

    return None

start = input("Enter the INITIAL state (give '0' for empty space): ")

if len(start) == 9 and set(start) == set('012345678'):
    print("INITIAL state:")
    print_state(start)

    result = dfs(start)

    if result is not None:
        print("Solution found!")
        print("Moves:", ' '.join(result))
        print("Number of moves:", len(result))

        current_state = start
        for i, move in enumerate(result, 1):
            current_state = move_tile(current_state, move)
            print(f"Move {i}: {move}")
            print_state(current_state)
    else:
        print("No solution exists for the given start state or max depth
reached.")
    else:
        print("Invalid input! Please enter a 9-digit string using digits 0-8
without repetition.")

print("1BM23CS292 Sameksha")

```

PROGRAM 3

Week 3

08/Sep

Date
Page

Apply A* Algorithm

Misplaced Tiles Manhattan Distance

2	8	3
1	6	4
7		5

1	2	3
8		4
7	6	5

Initial

Misplaced Tiles

Final

2	8	3
1	6	4
7		5

$g(n) = 1$
 $h(n) = 2$

R

2	8	3
1	6	4
7	5	

2	8	3
1	6	4
7	5	

2	8	3
1	6	4
7	5	5

$f(n) = 3 + 1 = 4$

$g(n) = 2$

2	8	3
3	1	4
7	6	5

2	8	3
1	4	
7	6	5

2	8	3
1	8	4
7	6	5

2	8	3
1	6	4
7	1	5

$f(n) = 5$

2	8	3
7	1	4
6	5	

2	8	3
7	6	5
1	4	

2	8	3
1	8	4
7	6	5

2	8	3
1	8	4
7	6	5

$f(n) = 7$

2	8	3
7	6	5
1	4	

2	8	3
7	6	5
1	4	

2	8	3
7	6	5
1	4	

2	8	3
7	6	5
1	4	

D

1	2	3
	8	4
7	6	5

1	2	3
8		4
7	6	5

$f(n) = 0 + 5 = 5$

1	2	3
7	8	4
	6	5

A* Algorithm

A* search evaluates nodes by combining $g(n)$, the cost to reach the node and $h(n)$, the cost to get from the node to the goal.

$$f(n) = g(n) + h(n)$$

$f(n)$ is the evaluation function which gives the cheapest solution cost.

$g(n)$ is the exact cost to reach node n from the initial state.

$h(n)$ is an estimation of the assumed cost from current state (n) to reach the goal.

Q
 $(2, 2) \rightarrow (5, 1)$
 $(1, 2) \rightarrow (2, 2)$
 $(1, 3) \rightarrow (2, 1)$

```

#misplaces tiles A*technique
#WEEK 3
import heapq

goal_state = '123804765'

moves = {
    'U': -3,
    'D': 3,
    'L': -1,
    'R': 1
}

invalid_moves = {
    0: ['U', 'L'], 1: ['U'], 2: ['U', 'R'],
    3: ['L'], 5: ['R'],
    6: ['D', 'L'], 7: ['D'], 8: ['D', 'R']
}

def move_tile(state, direction):
    index = state.index('0')
    if direction in invalid_moves.get(index, []):
        return None

    new_index = index + moves[direction]
    if new_index < 0 or new_index >= 9:
        return None

    state_list = list(state)
    state_list[index], state_list[new_index] = state_list[new_index],
state_list[index]
    return ''.join(state_list)

def print_state(state):
    for i in range(0, 9, 3):
        print(' '.join(state[i:i+3]).replace('0', ' '))
    print()

def misplaced_tiles(state):
    """Heuristic: count of tiles not in their goal position (excluding
zero)."""
    return sum(1 for i, val in enumerate(state) if val != '0' and val !=
goal_state[i])

def a_star(start_state):
    visited_count = 0

```

```

open_set = []
heapq.heappush(open_set, (misplaced_tiles(start_state), 0, start_state,
[]))
visited = set()

while open_set:
    f, g, current_state, path = heapq.heappop(open_set)
    visited_count += 1

    if current_state == goal_state:
        return path, visited_count

    if current_state in visited:
        continue
    visited.add(current_state)

    for direction in moves:
        new_state = move_tile(current_state, direction)
        if new_state and new_state not in visited:
            new_g = g + 1
            new_f = new_g + misplaced_tiles(new_state)
            heapq.heappush(open_set, (new_f, new_g, new_state, path +
[direction]))

    return None, visited_count

# Main
start = input("Enter start state (e.g., 724506831): ")

if len(start) == 9 and set(start) == set('012345678'):
    print("Start state:")
    print_state(start)

    result, visited_states = a_star(start)

    print(f"Total states visited: {visited_states}")

    if result is not None:
        print("Solution found!")
        print("Moves:", ' '.join(result))
        print("Number of moves:", len(result))
        print("SAMEKSHA 1BM23CS292\n")

    current_state = start
    for i, move in enumerate(result, 1):
        current_state = move_tile(current_state, move)

```

```

        print(f"Move {i}: {move}")
        print_state(current_state)
    else:
        print("No solution exists for the given start state.")
else:
    print("Invalid input! Please enter a 9-digit string using digits 0-8
without repetition.")

```

output:

Enter start state (e.g., 724506831): 124305678

Start state:

```

1 2 4
3   5
6 7 8

```

Total states visited: 789

Solution found!

Moves: L U R D D R U U L L D R D L U R

Number of moves: 16

SAMEKSHA 1BM23CS292

Move 1: L

```

1 2 4
   3 5
6 7 8

```

Move 2: U

```

   2 4
1 3 5
6 7 8

```

Move 3: R

```

2   4
1 3 5
6 7 8

```

Move 4: D

```

2 3 4
1   5
6 7 8

```

Move 5: D

```

2 3 4
1 7 5
6   8

```

Move 6: R

```

2 3 4
1 7 5
6 8

```

Move 7: U

```

2 3 4
1 7

```

6 8 5

Move 8: U

2 3

1 7 4

6 8 5

Move 9: L

2 3

1 7 4

6 8 5

Move 10: L

2 3

1 7 4

6 8 5

Move 11: D

1 2 3

7 4

6 8 5

Move 12: R

1 2 3

7 4

6 8 5

Move 13: D

1 2 3

7 8 4

6 5

Move 14: L

1 2 3

7 8 4

6 5

Move 15: U

1 2 3

8 4

7 6 5

Move 16: R

1 2 3

8 4

7 6 5

Program 4

Implement Hill Climbing search algorithm and Hill climbing algorithm

Week 4.

15/09/25

Hill climbing Search Algorithm to solve
N-Queens Problem
(4) Queens.

Input :

			Q
	Q		
		Q	
Q			

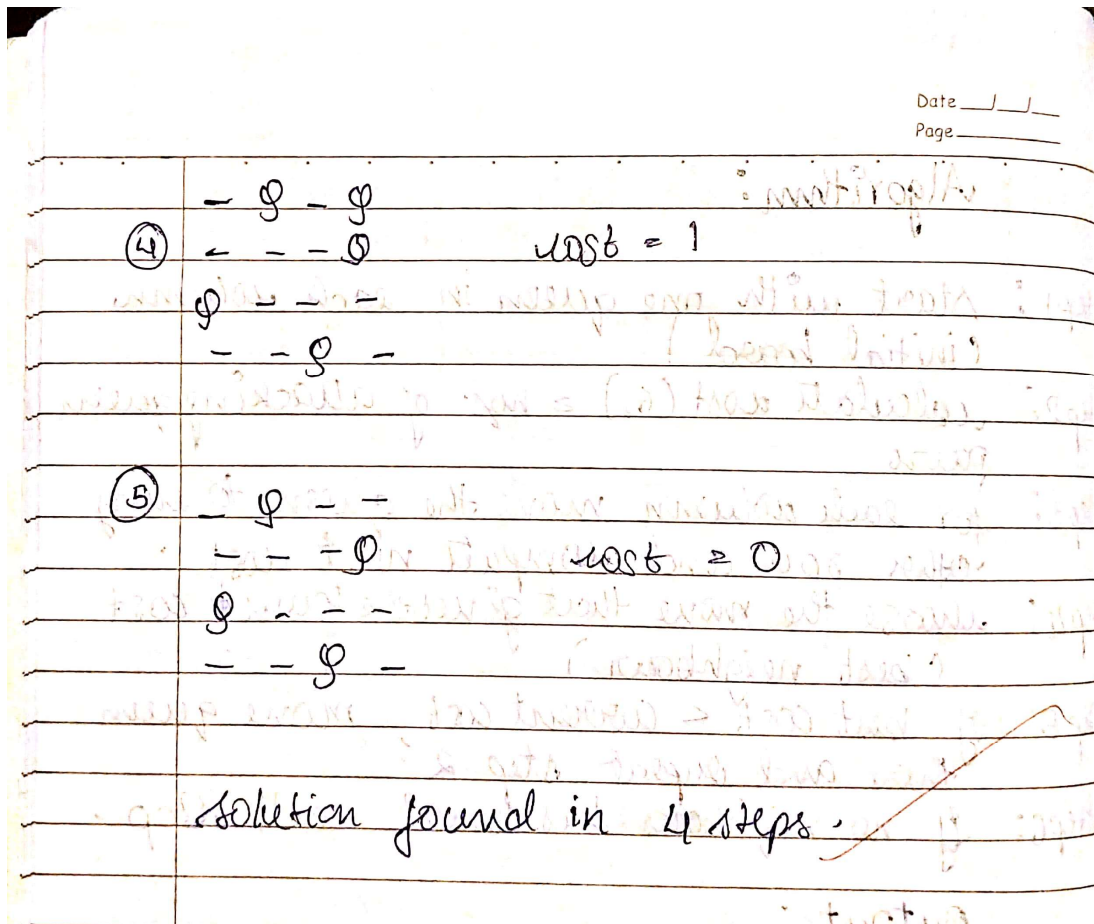
1> - - - Q (2, 2) ↔ (3, 3)
- Q - - (1, 4) ↔ (4, 1)
- - Q -
Q - - - cost = 2

2> - - - - (2, 2) ↔ (2, 4)
- Q - - (2, 2) ↔ (3, 3)
- - Q - (3, 3) ↔ (2, 4)
Q - - - cost = 3

3> - - Q - (1, 3) ↔ (2, 2)
- Q - Q (2, 2) ↔ (2, 4)
- - - - (1, 3) ↔ (2, 4)
Q - - - cost = 3

4> - - - - (2, 4) ↔ (3, 3)
- - - Q (3, 2) ↔ (3, 1)
- Q Q - (3, 2) ↔ (4, 1)
Q - - - cost = 3

5> - Q - -
- - - Q cost = 0
Q - - -
- - Q -



Code:

```
import random
import time

def print_board(state):
    """Prints the chessboard for a given state."""
    n = len(state)
    for row in range(n):
        line = ""
        for col in range(n):
            if state[col] == row:
                line += "Q "
            else:
                line += ". "
        print(line)
    print()

def compute_heuristic(state):
    """Computes the number of attacking pairs of queens."""
    h = 0
    n = len(state)
```

```

        for i in range(n):
            for j in range(i + 1, n):
                if state[i] == state[j] or abs(state[i] - state[j]) == abs(i -
j):
                    h += 1
        return h

def get_neighbors(state):
    """Generates all possible neighbors by moving one queen in its column."""
    neighbors = []
    n = len(state)
    for col in range(n):
        for row in range(n):
            if row != state[col]:
                neighbor = list(state)
                neighbor[col] = row
                neighbors.append(neighbor)
    return neighbors

def hill_climb_verbose(initial_state, step_delay=0.5):
    """Hill climbing algorithm with verbose output at each step."""
    current = initial_state
    current_h = compute_heuristic(current)
    step = 0

    print(f"Initial state (heuristic: {current_h}):")
    print_board(current)
    time.sleep(step_delay)

    while True:
        neighbors = get_neighbors(current)
        next_state = None
        next_h = current_h

        for neighbor in neighbors:
            h = compute_heuristic(neighbor)
            if h < next_h:
                next_state = neighbor
                next_h = h

        if next_h >= current_h:
            print(f"Reached local minimum at step {step}, heuristic:
{current_h}")
            return current, current_h

        current = next_state

```

```

        current_h = next_h
        step += 1
        print(f"Step {step}: (heuristic: {current_h})")
        print_board(current)
        time.sleep(step_delay)

def solve_n_queens_verbose(n, max_restarts=1000):
    """Solves N-Queens problem using hill climbing with restarts and verbose
    output."""
    for attempt in range(max_restarts):
        print(f"\n=== Restart {attempt + 1} ===\n")
        initial_state = [random.randint(0, n - 1) for _ in range(n)]
        solution, h = hill_climb_verbose(initial_state)
        if h == 0:
            print(f"✔ Solution found after {attempt + 1} restart(s):")
            print_board(solution)
            return solution
        else:
            print(f"✗ No solution in this attempt (local minimum).\n")
    print("Failed to find a solution after max restarts.")
    return None

# --- Run the algorithm ---
if __name__ == "__main__":
    N = int(input("Enter the number of queens (N): "))
    solve_n_queens_verbose(N)
    print("1BM23CS292 sameksha")

```

output:

Enter the number of queens (N): 4

=== Restart 1 ===

Initial state (heuristic: 4):

```

. . . .
. . Q Q
. Q . .
Q . . .

```

Step 1: (heuristic: 2)

```

Q . . .
. . Q Q
. Q . .
. . . .

```

Step 2: (heuristic: 1)

```

Q . . .
. . . Q
. Q . .
. . Q .

```

Reached local minimum at step 2, heuristic: 1
✘ No solution in this attempt (local minimum).

=== Restart 2 ===

Initial state (heuristic: 3):

```
. . . .  
. Q Q .  
. . . Q  
Q . . .
```

Step 1: (heuristic: 1)

```
. . Q .  
. Q . .  
. . . Q  
Q . . .
```

Reached local minimum at step 1, heuristic: 1
✘ No solution in this attempt (local minimum).

=== Restart 3 ===

Initial state (heuristic: 4):

```
. . Q Q  
. . . .  
Q Q . .  
. . . .
```

Step 1: (heuristic: 2)

```
. . Q .  
. . . .  
Q Q . .  
. . . Q
```

Step 2: (heuristic: 1)

```
. . Q .  
Q . . .  
. Q . .  
. . . Q
```

Reached local minimum at step 2, heuristic: 1
✘ No solution in this attempt (local minimum).

=== Restart 4 ===

Initial state (heuristic: 3):

```
Q . . Q  
. . . .  
. . Q .  
. Q . .
```

Step 1: (heuristic: 1)

```
. . . Q  
Q . . .
```

```

. . Q .
. Q . .

Reached local minimum at step 1, heuristic: 1
✗ No solution in this attempt (local minimum).

```

=== Restart 5 ===

Initial state (heuristic: 5):

```

. . . .
. . . .
Q . Q Q
. Q . .

```

Step 1: (heuristic: 2)

```

. . . .
Q . . .
. . Q Q
. Q . .

```

Step 2: (heuristic: 0)

```

. . Q .
Q . . .
. . . Q
. Q . .

```

Reached local minimum at step 2, heuristic: 0

✓ Solution found after 5 restart(s):

```

. . Q .
Q . . .
. . . Q
. Q . .

```

1BM23CS292 sameksha

Problem 5

Simulated Annealing to Solve 8-Queens problem

Week 5

Simulated Annealing

Algorithm:

current $\leftarrow$ initial state

$T \leftarrow$ a large positive value

while $T > 0$ do

 next $\leftarrow$ a random neighbor
 of current

$\Delta E \leftarrow$ current cost - next cost

 if $\Delta E \geq 0$ then

 current $\leftarrow$ next

 else

 current $\leftarrow$ next with
 probability $p = e^{-\Delta E/T}$

end if

Code:

```
#WEEK 5
import random
import math

def compute_heuristic(state):
    """Number of attacking pairs."""
    h = 0
    n = len(state)
    for i in range(n):
        for j in range(i + 1, n):
            if state[i] == state[j] or abs(state[i] - state[j]) == abs(i - j):
                h += 1
    return h

def random_neighbor(state):
    """Returns a neighbor by randomly changing one queen's row."""
    n = len(state)
    neighbor = state[:]
    col = random.randint(0, n - 1)
    old_row = neighbor[col]
    new_row = random.choice([r for r in range(n) if r != old_row])
    neighbor[col] = new_row
```



```

    return neighbor

def dual_simulated_annealing(n, max_iter=10000, initial_temp=100.0,
cooling_rate=0.99):
    """Simulated Annealing with dual acceptance strategy."""
    current = [random.randint(0, n - 1) for _ in range(n)]
    current_h = compute_heuristic(current)
    temperature = initial_temp

    for step in range(max_iter):
        if current_h == 0:
            print(f"✔ Solution found at step {step}")
            return current

        neighbor = random_neighbor(current)
        neighbor_h = compute_heuristic(neighbor)
        delta = neighbor_h - current_h

        if delta < 0:
            current = neighbor
            current_h = neighbor_h
        else:
            # Dual acceptance: standard + small chance of higher uphill move
            probability = math.exp(-delta / temperature)
            if random.random() < probability:
                current = neighbor
                current_h = neighbor_h

        temperature *= cooling_rate
        if temperature < 1e-5: # Restart if stuck
            temperature = initial_temp
            current = [random.randint(0, n - 1) for _ in range(n)]
            current_h = compute_heuristic(current)

    print("✗ Failed to find solution within max iterations.")
    return None

# --- Run the algorithm ---
if __name__ == "__main__":
    N = int(input("Enter number of queens (N): "))
    solution = dual_simulated_annealing(N)

    if solution:
        print("Position format:")
        print("[", " ".join(str(x) for x in solution), "]")
        print("Heuristic:", compute_heuristic(solution))

```

```
print("1BM23CS292 sameksha")
```

output:

```
Enter number of queens (N): 4
✓ Solution found at step 18
Position format:
[ 1 3 0 2 ]
Heuristic: 0
1BM23CS292 sameksha
```

Program 6

Create a knowledge base using propositional logic and show that the given query entails the knowledge base or not

Week-6 22/09/23

Propositional Logic -

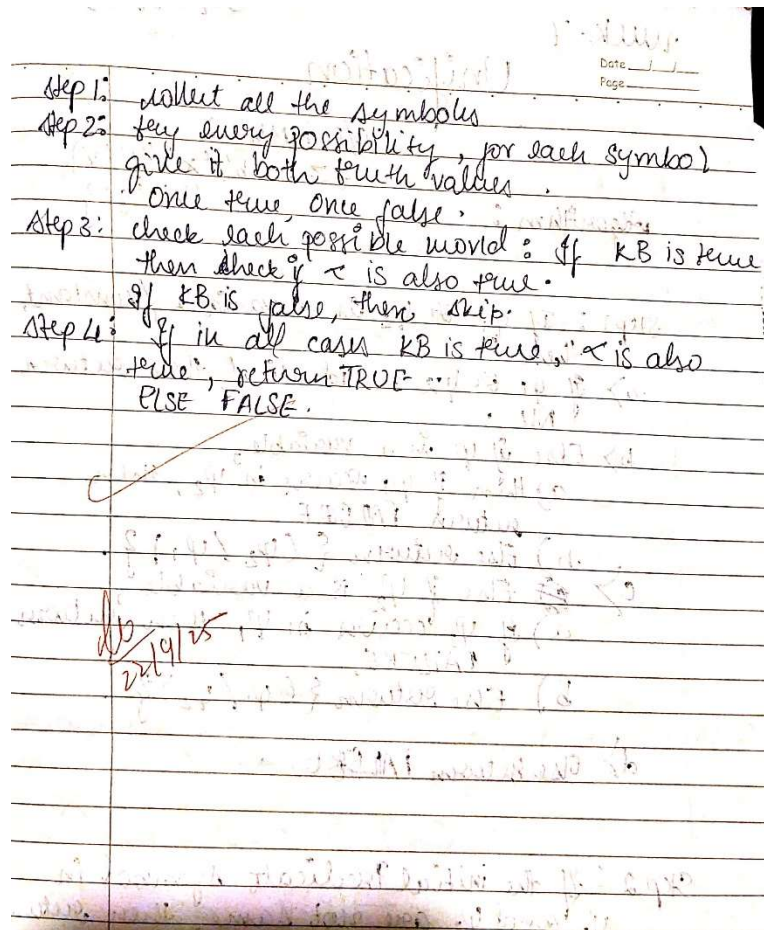
Truth tables for connectives

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$
F	F	T	F	F	T
F	T	T	F	T	F
T	F	F	F	T	F
T	T	F	T	T	T

g> Propositional Inference : Enumeration Method

$\alpha = A \vee B$ $KB = (A \vee C) \wedge (B \vee \neg C)$

A	B	C	$A \vee C$	$B \vee \neg C$	KB	α
F	F	F	F	T	F	F
F	F	T	T	F	F	F
F	T	F	F	T	F	T
F	T	T	T	T	T	T
T	F	F	T	T	T	T
T	F	T	T	F	F	T
T	T	F	T	T	T	T
T	T	T	T	T	T	T



Code:

```
#WEEK7
import itertools
import pandas as pd
from tabulate import tabulate
import re

# Helper: convert booleans to T/F
def tf(val):
    return "T" if val else "F"

# Replace logical symbols (V, ^, ¬) with Python equivalents
def translate(expr: str) -> str:
    return (
        expr.replace("V", " or ")
        .replace("^", " and ")
        .replace("¬", " not ")
    )

# == Take KB and alpha (query) from user ==
kb_expr = input("Enter your Knowledge Base (use ^, V, ¬): ")
```

```

alpha_expr = input("Enter your  $\alpha$  (query) (use  $\wedge$ ,  $\vee$ ,  $\neg$ ): ")

# Translate to Python syntax
kb_py = translate(kb_expr)
alpha_py = translate(alpha_expr)

# === Detect variables dynamically (any single letter, uppercase/lowercase)
===
vars_in_expr = sorted(set(re.findall(r"\b[a-zA-Z]\b", kb_expr + alpha_expr)))

if not vars_in_expr:
    raise ValueError("No variables detected. Please use single letters like
A, b, C...")

# Generate truth table
rows = []
for values in itertools.product([False, True], repeat=len(vars_in_expr)):
    local_vars = dict(zip(vars_in_expr, values))
    kb_val = eval(kb_py, {}, local_vars)
    alpha_val = eval(alpha_py, {}, local_vars)

    row = {var: tf(val) for var, val in local_vars.items()}
    row["KB"] = tf(kb_val)
    row[f" $\alpha$  = {alpha_expr}"] = tf(alpha_val)
    rows.append(row)

# Convert to DataFrame
df = pd.DataFrame(rows)

# Full truth table
print("\n=== Full Truth Table ===")
print(tabulate(df, headers="keys", tablefmt="fancy_grid", showindex=False))

# Filtered truth table (only KB = T)
filtered_df = df[df["KB"] == "T"]

print("\n=== Rows where KB is True ===")
if not filtered_df.empty:
    print(tabulate(filtered_df, headers="keys", tablefmt="fancy_grid",
showindex=False))
else:
    print("No rows where KB is True (KB is unsatisfiable).")

# Entailment check
query_col = f" $\alpha$  = {alpha_expr}"

```

```

entails = all(filtered_df[query_col] == "T") if not filtered_df.empty else
True
print(f"\nDoes KB entail  $\alpha$  ({alpha_expr})? ->", "Yes" if entails else "No")
print("SAMEIKSHA 1BM23CS292\n")

```

output:

Enter your Knowledge Base (use $\wedge$, $\vee$, $\neg$): $(a \vee c) \wedge (b \vee \neg c)$
Enter your α (query) (use $\wedge$, $\vee$, $\neg$): $a \vee b$

=== Full Truth Table ===

a	b	c	KB	$\alpha = a \vee b$
F	F	F	F	F
F	F	T	F	F
F	T	F	F	T
F	T	T	T	T
T	F	F	T	T
T	F	T	F	T
T	T	F	T	T
T	T	T	T	T

=== Rows where KB is True ===

a	b	c	KB	$\alpha = a \vee b$
F	T	T	T	T
T	F	F	T	T
T	T	F	T	T
T	T	T	T	T

Does KB entail α ($a \vee b$)? -> Yes
SAMEIKSHA 1BM23CS292

Program 8

Unification using FOL

mmk-7

30/09/25

Date / /

Page

Unification

O/P : MGU :

$\{Z :: 'b', 'x' :: ('f', 'y'), 'y' :: 'g'\}$

Algorithm :

Step 1 : If φ_1 or φ_2 is a variable / constant then :

- a) If φ_1 or φ_2 are identical, then return NIL.
- b) Else if φ is a variable,
 - a) then if φ_1 occurs in φ_2 , then return FAILURE
 - b) Else return $\{(\varphi_2 / \varphi_1)\}$.
- c) ~~Else~~ Else if φ_2 is a variable,
 - a) If φ_2 occurs in φ_1 , then return FAILURE,
 - b) Else return $\{(\varphi_1 / \varphi_2)\}$.
- d) Else return FAILURE.

Step 2 : If the initial Predicate symbol in φ_1 and φ_2 are not same, then return FAILURE.

Step 3 : If φ_1 and φ_2 have a different number of arguments, then return FAILURE.

Step 4 : Set Substitution set (SUBST) to NIL.

step 5: For $i=1$ to the number of elements in V_1 .

a) Call unify function with the i th element of V_1 and i th element of V_2 , and put the result into S .

b) If $S = \text{failure}$ then returns FAILURE

c) If $S \neq \text{NIL}$ then do,

a) Apply S to the remainder of both L_1 and L_2 .

b) $\text{SUBST} = \text{APPEND}(S, \text{SUBST})$.

step 6: Return SUBST.

Q1) Unify ($\text{know}(\text{John}, x)$, $\text{knows}(\text{John}, \text{Janet})$)

$\theta = x/\text{Janet}$

Q2) Unify ($\text{knows}(\text{John}, x)$, $\text{knows}(y, \text{Bill})$)

$\theta = y/\text{John}$

Q3) Find MGV of $\{P(b, x, f(g(z)))\}$ & $\{P(z, f(y), f(y))\}$

$z \leftrightarrow b$ $g(y) \leftrightarrow x$ $y \leftrightarrow z$

Q4) Find MGV of $\{g(a, g(x, a), f(y))\}$ & $\{g(a, g(f(b), a), x)\}$

$x \leftrightarrow f(b)$ $x \leftrightarrow f(g)$

Q5) Unify $\text{prime}(11)$ and $\text{prime } y$

$y \leftrightarrow 11$

Q6) Unify $\text{knows}(\text{John}, x)$, $\text{knows}(y, \text{mother}(x))$

$x \leftrightarrow \text{mother}(y)$ $y \leftrightarrow \text{mother}(\text{John})$

Code:

```
class UnificationError(Exception):
    pass

def occurs_check(var, term):
    """Check if a variable occurs in a term (to prevent infinite
    recursion)."""
    if var == term:
        return True
    if isinstance(term, tuple): # Term is a compound (function term)
        return any(occurs_check(var, subterm) for subterm in term)
    return False

def unify(term1, term2, substitutions=None):
    """Try to unify two terms, return the MGU (Most General Unifier)."""
    if substitutions is None:
        substitutions = {}

    # If both terms are equal, no further substitution is needed
    if term1 == term2:
        return substitutions

    # If term1 is a variable, we substitute it with term2
    elif isinstance(term1, str) and term1.isupper():
        # If term1 is already substituted, recurse
        if term1 in substitutions:
            return unify(substitutions[term1], term2, substitutions)
        elif occurs_check(term1, term2):
            raise UnificationError(f"Occurs check fails: {term1} in {term2}")
        else:
            substitutions[term1] = term2
            return substitutions

    # If term2 is a variable, we substitute it with term1
    elif isinstance(term2, str) and term2.isupper():
        # If term2 is already substituted, recurse
        if term2 in substitutions:
            return unify(term1, substitutions[term2], substitutions)
        elif occurs_check(term2, term1):
            raise UnificationError(f"Occurs check fails: {term2} in {term1}")
        else:
            substitutions[term2] = term1
            return substitutions
```



```

    # If both terms are compound (i.e., functions), unify their parts
    recursively
    elif isinstance(term1, tuple) and isinstance(term2, tuple):
        # Ensure that both terms have the same "functor" and number of
        arguments
        # if len(term1) != len(term2):
        #     raise UnificationError(f"Function arity mismatch: {term1} vs
        {term2}")

        for subterm1, subterm2 in zip(term1, term2):
            substitutions = unify(subterm1, subterm2, substitutions)

        return substitutions

    else:
        raise UnificationError(f"Cannot unify: {term1} with {term2}")

# Define the terms as tuples
term1 = ('p', 'b', 'X', ('f', ('g', 'Z')))
term2 = ('p', 'Z', ('f', 'Y'), ('f', 'Y'))

try:
    # Find the MGU
    result = unify(term1, term2)
    print("Most General Unifier (MGU):")
    print(result)
except UnificationError as e:
    print(f"Unification failed: {e}")
print("Sameksha 1bm23cs292")

```

output:

```

Most General Unifier (MGU):
{'Z': 'b', 'X': ('f', 'Y'), 'Y': ('g', 'Z')}
Sameksha 1BM23CS292

```

Program 8

Create a knowledge base consisting of first order logic statements and prove the given query using forward reasoning.

Week-8

13/10/25

Date _____
Page _____

First Order Logic

Create a knowledge base consisting of first order logic statements and prove the given query using forward reasoning.

Premises

Rules

- $P \Rightarrow Q$ - conclusion
- $L \wedge M \Rightarrow P$
- $B \wedge L \Rightarrow M$
- $A \wedge P \Rightarrow L$
- $A \wedge B \Rightarrow L$

fact

- A
- B

Prove Q

```
graph TD
    A((A)) --- L((L))
    B((B)) --- L
    L --- M((M))
    M --- P((P))
    P --- Q((Q))
```

13/10/25

Date _____
Page _____

Date _____
Page _____

sitting at
rel
forward

The law says that it is a crime for an American to sell weapons to hostile nations. The country NoNo, an enemy of America, has some missiles, and all of its missiles were sold to it by Colonel West, who is American. An enemy of America counts as "hostile".

→ Prove that "west is criminal".

Rules

① $\forall x, y, z \text{ American}(x) \wedge \text{Weapon}(y) \wedge \text{Sells}(x, y, z) \Rightarrow \text{Criminal}(x)$

2) $\text{Owns}(\text{NoNo}, M1)$

3) $\text{Missile}(M1)$

4) $\forall x \text{ Missile}(x) \Rightarrow \dots$

Facts

$\forall x \text{ Missile}(x) \wedge \text{Owns}(\text{NoNo}, x)$

$\text{American}(\text{West})$

$\Rightarrow \text{Sell}(\text{West}, x, \text{NoNo})$

$\text{Enemy}(\text{NoNo}, \text{America})$

①

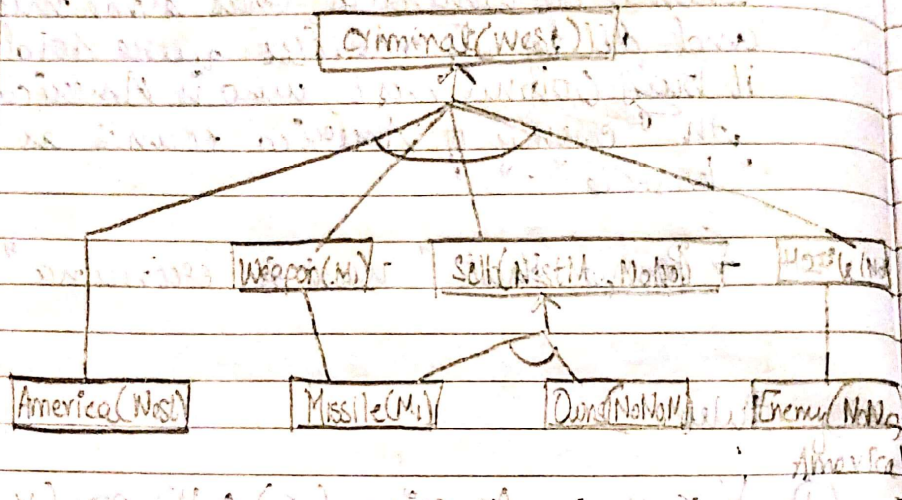
$\text{Owns}(\text{NoNo}, M1)$

$\forall x \text{ Enemy}(x, \text{America}) \Rightarrow \text{Hostile}(x)$

$\text{Missile}(M1)$

$\forall x \text{ Missile}(x) \Rightarrow \text{Weapon}(x)$

$American(x) \wedge weapon(y) \wedge sells(x, y, z)$
 $\wedge Hostile(z) \Rightarrow Criminal(x)$



Algorithm:

function FOL-FC-Ask(KB, α) returns
a substitution or false

inputs : KB, the knowledge base, a set of
first-order definite clauses
 α , the query, an atomic sentence

local variables : new, the new sentences
inferred in each iteration

repeat until new is empty

new $\leftarrow \emptyset$

for each rule in KB do

$(p_1 \wedge \dots \wedge p_n \Rightarrow q) \leftarrow \text{STANDARDIZE-}$
 $\text{VARIABLES}(\text{rule})$

P-31000

Date ___/___/___

Page ___

for each θ such that $\text{SUBST}(\theta, p_1 \wedge \dots \wedge p_n)$
 $= \text{SUBST}(\theta, p'_1 \wedge \dots \wedge p'_n)$
for some $p'_1, \dots, p'_n$ in KB
 $q' \leftarrow \text{SUBST}(\theta, q)$
if q' does not unify with some sentence
already in KB or new then add q' to
new
 $\emptyset \leftarrow \text{UNIFY}(q', x)$
if $\emptyset$ is not fail then return $\emptyset$
add new to KB
return False.

Output:

Robert is Criminal.

WJW
13-1-20

Code:

```
#WEEK 8
#FIRST ORDER LOGIC
# Define the knowledge base
facts = {
    'American(Robert)': True, # Robert is an American
    'Hostile(A)': True,      # Country A is hostile to America
    'Sells_Weapons(Robert, A)': True # Robert sold weapons to Country A
}

# Define the law/rule: If American(X) and Hostile(Y) and Sells_Weapons(X, Y),
then Crime(X)
def forward_reasoning(facts):
    # Apply the rule: If American(X) and Hostile(Y) and Sells_Weapons(X, Y),
    then Crime(X)
    if facts.get('American(Robert)', False) and facts.get('Hostile(A)',
False) and facts.get('Sells_Weapons(Robert, A)', False):
        facts['Crime(Robert)'] = True # Robert is a criminal

# Perform forward reasoning to see if we can deduce that Robert is a criminal
forward_reasoning(facts)

# Output the result based on the fact derived
if facts.get('Crime(Robert)', False):
    print("Robert is a criminal.")
else:
    print("Robert is not a criminal.")
print("Sameksha 1BM23CS292")
```

output:

```
Robert is a criminal.
Sameksha 1BM23CS292
```


Program 9

Create a knowledge base consisting of first order logic statements and prove the given query using Resolution

Week-9

Date: __/__/__
Page: ____

FOL using Resolution

~~Eliminate~~ Algorithm for Resolution

- Convert all sentences to CNF
- Negate conclusion S and convert result to CNF
- Add negated conclusion S to the premise clauses
- Repeat until contradiction / no progress is made:
 - a) Select 2 clauses (call them parent clauses)
 - b) Resolve them together, performing all required unifications.
 - c) If resolvent is the empty clause, a contradiction has been found.
 - d) If not, add resolvent to the premises.

Output:

Knowledge Base CNF Clauses:

$\sim \text{Parent}(x, y) \mid \text{Ancestor}(x, y)$

$\text{Parent}(\text{John}, \text{Mary})$

$\text{Parent}(\text{Mary}, \text{Sam})$

$\sim \text{Parent}(x, y) \mid \sim \text{Ancestor}(y, z) \mid \text{Ancestor}(x, z)$

Query: $\text{Ancestor}(\text{John}, \text{Sam}) \Rightarrow \text{TRUE}$

CODE:

```
import re
import copy

# =====
# Predicate Class
# =====

class Predicate:
    def __init__(self, predicate_string):
        self.predicate_string = predicate_string
        self.name, self.arguments, self.negative =
self.parse_predicate(predicate_string)

    def parse_predicate(self, predicate_string):
        neg = predicate_string.startswith('~')
        if neg:
            predicate_string = predicate_string[1:]
        m = re.match(r"([A-Za-z_][A-Za-z0-9_]*)\((.*?)\)", predicate_string)
        if not m:
            raise ValueError(f"Invalid predicate: {predicate_string}")
        name, args = m.groups()
        args = [a.strip() for a in args.split(",")]
        return name, args, neg

    def negate(self):
        self.negative = not self.negative
        if self.predicate_string.startswith('~'):
            self.predicate_string = self.predicate_string[1:]
        else:
            self.predicate_string = '~' + self.predicate_string

    def unify_with_predicate(self, other):
        """Attempt to unify two predicates; return substitution dict or
False."""
        if self.name != other.name or len(self.arguments) !=
len(other.arguments):
            return False
        subs = {}
        for a, b in zip(self.arguments, other.arguments):
            if a == b:
                continue
            if a[0].islower():
                subs[a] = b
            elif b[0].islower():
                subs[b] = a
```

```

        else:
            return False
    return subs

def substitute(self, subs):
    """Apply substitution dictionary."""
    self.arguments = [subs.get(a, a) for a in self.arguments]
    self.predicate_string = (
        ('~' if self.negative else '') +
        self.name + '(' + ','.join(self.arguments) + ')'
    )

def __repr__(self):
    return self.predicate_string

# =====
# Statement Class (CNF Clause)
# =====
class Statement:
    def __init__(self, statement_string):
        self.statement_string = statement_string
        self.predicate_set = self.parse_statement(statement_string)

    def parse_statement(self, statement_string):
        parts = statement_string.split('|')
        predicates = []
        for p in parts:
            predicates.append(Predicate(p.strip()))
        return set(predicates)

    def add_statement_to_KB(self, KB, KB_HASH):
        KB.add(self)
        for predicate in self.predicate_set:
            key = predicate.name
            if key not in KB_HASH:
                KB_HASH[key] = set()
            KB_HASH[key].add(self)

    def get_resolving_clauses(self, KB_HASH):
        resolving_clauses = set()
        for predicate in self.predicate_set:
            key = predicate.name
            if key in KB_HASH:
                resolving_clauses |= KB_HASH[key]
        return resolving_clauses

```

```

def resolve(self, other):
    """Resolve two statements; return new derived statements or False if
    contradiction."""
    new_statements = set()
    for p1 in self.predicate_set:
        for p2 in other.predicate_set:
            if p1.name == p2.name and p1.negative != p2.negative:
                subs = p1.unify_with_predicate(p2)
                if subs is False:
                    continue
                new_pred_set = set()
                for pred in
self.predicate_set.union(other.predicate_set):
                    if pred not in (p1, p2):
                        pred_copy = copy.deepcopy(pred)
                        pred_copy.substitute(subs)
                        new_pred_set.add(pred_copy)
                if not new_pred_set:
                    return False # contradiction
                new_stmt = Statement(''.join(sorted([str(p) for p in
new_pred_set])))
                new_statements.add(new_stmt)
    return new_statements

def __repr__(self):
    return self.statement_string

# =====
# Utility: Convert FOL Sentence to CNF Clauses
# =====
def fol_to_cnf_clauses(sentence):
    """
    Convert simple implications and conjunctions into CNF.
    Example:
        "A(x,y) => B(x,y)" becomes "~A(x,y) | B(x,y)"
        "A(x,y) & B(y,z) => C(x,z)" becomes "~A(x,y) | ~B(y,z) | C(x,z)"
    """
    sentence = sentence.replace(' ', '')

    if '=>' in sentence:
        lhs, rhs = sentence.split('=>')
        parts = lhs.split('&')
        negated_lhs = ['~' + p for p in parts]
        disjunction = ''.join(negated_lhs + [rhs])

```

```

        return [disjunction]

# Split conjunctions into separate clauses
if '&' in sentence:
    return sentence.split('&')

return [sentence]

# =====
# Knowledge Base & Resolution Logic
# =====
KILL_LIMIT = 8000

def prepare_knowledgebase(fol_sentences):
    KB = set()
    KB_HASH = {}
    for sentence in fol_sentences:
        clauses = fol_to_cnf_clauses(sentence)
        for clause in clauses:
            stmt = Statement(clause)
            stmt.add_statement_to_KB(KB, KB_HASH)
    return KB, KB_HASH

def FOL_Resolution(KB, KB_HASH, query):
    KB2 = set()
    query.add_statement_to_KB(KB2, KB_HASH)
    query.add_statement_to_KB(KB, KB_HASH)
    while True:
        new_statements = set()
        if len(KB) > KILL_LIMIT:
            return False
        for s1 in KB:
            for s2 in s1.get_resolving_clauses(KB_HASH):
                if s1 == s2:
                    continue
                resolvents = s1.resolve(s2)
                if resolvents is False:
                    return True
                new_statements |= resolvents
        if new_statements.issubset(KB):
            return False
        new_statements -= KB
        KB |= new_statements

```

```

# =====
# Example Knowledge Base & Query
# =====
def main():
    fol_sentences = [
        "Parent(John, Mary)",
        "Parent(Mary, Sam)",
        "Parent(x, y) => Ancestor(x, y)",
        "Parent(x, y) & Ancestor(y, z) => Ancestor(x, z)"
    ]
    queries = ["Ancestor(John, Sam)"]

    KB, KB_HASH = prepare_knowledgebase(fol_sentences)

    print("\nKnowledge Base CNF Clauses:")
    for stmt in KB:
        print("  ", stmt)

    for query_str in queries:
        query_predicate = Predicate(query_str)
        query_predicate.negate()
        query_stmt = Statement(str(query_predicate))
        satisfiable = FOL_Resolution(copy.deepcopy(KB),
        copy.deepcopy(KB_HASH), query_stmt)
        print(f"\nQuery: {query_str} =>", "TRUE" if satisfiable else "FALSE")

if __name__ == "__main__":
    main()

```

OUTPUT:

```

Knowledge Base CNF Clauses:
  Parent(Mary, Sam)
  Parent(John, Mary)
  ~Parent(x, y) | Ancestor(x, y)
  ~Parent(x, y) | ~Ancestor(y, z) | Ancestor(x, z)

Query: Ancestor(John, Sam) => TRUE

```

PROGRAM 10

Implement Alpha-Beta Pruning.

Week 10

Date: ____/____/____
Page: ____

Alpha Beta Pruning.

Algorithm :

function ALPHA-BETA-SEARCH (state)
returns an action
 $v \leftarrow \text{MAX-VALUE}(\text{state}, -\infty, +\infty)$
returns the action in Actions (state)
with value v .

function MAX-VALUE (state, α , β) returns a
utility value
if TERMINAL-TEST (state) then return
UTILITY (state)
 $v \leftarrow -\infty$
for each a in ACTIONS (state) do
 $v \leftarrow \text{MAX}(v, \text{MIN-VALUE}(\text{RESULT}(s, a),$
 $\alpha, \beta))$
if $v \geq \beta$ then return v
 $\alpha \leftarrow \text{MAX}(\alpha, v)$
return v .

~~10/6/21/15~~ function MIN-VALUE (state, α , β) returns a
utility value
if Terminal-test (state) then return
UTILITY (state)
 $v \leftarrow +\infty$
for each a in ACTIONS (state) do
 $v \leftarrow \text{MIN}(v, \text{MAX-VALUE}(\text{RESULT}$
 $(s, a), \alpha, \beta))$
if $v \leq \alpha$ then return v
 $\beta \leftarrow \text{MIN}(\beta, v)$ return v .

CODE:

```
import math

def alpha_beta(node, depth, alpha, beta, maximizingPlayer, game_tree):
    """
    Alpha-Beta pruning search algorithm
    node: current node in game tree
    depth: current depth
    alpha: best value for maximizer
    beta: best value for minimizer
    maximizingPlayer: True if maximizer's turn
    game_tree: dictionary representing tree {node: children or value}
    """
    # If leaf node or depth 0, return its value
    if depth == 0 or isinstance(game_tree[node], int):
        return game_tree[node]

    if maximizingPlayer:
        maxEval = -math.inf
        for child in game_tree[node]:
            eval = alpha_beta(child, depth-1, alpha, beta, False, game_tree)
            maxEval = max(maxEval, eval)
            alpha = max(alpha, eval)
            if beta <= alpha:
                break # Beta cut-off
        return maxEval
    else:
        minEval = math.inf
        for child in game_tree[node]:
            eval = alpha_beta(child, depth-1, alpha, beta, True, game_tree)
            minEval = min(minEval, eval)
            beta = min(beta, eval)
            if beta <= alpha:
                break # Alpha cut-off
        return minEval

# Example game tree
game_tree = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': 3,
    'E': 5,
    'F': 2,
    'G': 9
}
```

```
}  
  
best_value = alpha_beta('A', depth=3, alpha=-math.inf, beta=math.inf,  
maximizingPlayer=True, game_tree=game_tree)  
print("Best value for maximizer:", best_value)
```

OUTPUT:

Best value for maximizer: 3